ADVANCED PROGRAMMING LANGUAGES

Spring 2026, CM 0632. MSc in Computer Science and Information Technology, Ca' Foscari University of Venice

Algebrainc Data Types in Programming Languages

We complete the discussion on algebraic datatypes with a deeper dive on their implementation in different programming languages

Nominal vs Structural Typing

Following a common practice in type theoretic datatypes presentations, so far we have adopted a purely *structural* presentation that introduces new type expressions reflecting the structure of the datatypes the type expressions are associated with.

By contrast, programming languages most often adopt a different approach, known as *nominal typing*. In a nominal type system:

- every compound type must be explicitly introduced by a declaration before being used;
- each type name must be declared at most once, and following their declarations, types must always be referred to by their names

Nominal type systems have both advantages and disadvantages over structural presentations.

- The most significant advantage is probably that the type names in nominal systems are useful not only during type-checking, but at run time as well. Most nominal languages tag each run-time object with a header containing its type name, represented concretely as a pointer to a run-time data structure describing the type. These type tags are handy for a variety of purposes, including run-time type testing (e.g., Java's *instanceOf* tests and *downcasting*), printing, marshaling data structures into binary forms, . . . etc.

- A less essential, but pleasant, property of nominal systems is that, as we have discussed, they offer a natural and intuitive account of recursive types. Such types are ubiquitous in serious programming, being required to describe such common structures as lists and trees. Nominal type systems support them in the most straightforward possible way: referring to List in the body of its own declaration is just as easy as referring to any other type.

Structural systems, in turn, are appealing in that they are somewhat tidier and more elegant.

- In a structural setting, a type expression is a closed entity: it carries with it all the information that is needed to understand its meaning (by contrast, in a nominal system, we are always working with respect to some global collection of type name-definition pairs).

- Recursive types can also be handled structurally, as we have discussed, by introducing type-level recursive operators (e.g. fix or rec operators on types), though capturing complex schemes of mutually recursive definitions becomes somewhat heavy with such operators.

All in all, the usefulness of type tags and the simple treatment of recursive types are decisive benefits for the practice of programming, and hence it is no surprise to find that nominal type systems are the norm in mainstream programming languages, with few exceptions.

ADTs in Haskell

Record types. Haskell record types are product types with additional syntax to provide a convenient naming (and access) scheme for the product positions. For example the following is the Haskell version of the datatypes defined earlier:

```
data Pair = Pair { fst :: Int, sdn:: Int }
data Address = Addr { city :: String, street :: String, number :: Int }

let p = Pair { fst = 5, snd = 3 }
let a = Addr { city = "Boston", street = "Worcester", number = 15 }
```

Both types are introduced by a `data` declaration, which defines the new type name associated with the type being defined. Each new type comes with a data named data constructor to build the elements of the type's domain: for the type `Pair` the type and data constructors have the same name, whereas for addresses `Address` is the type constructor, while `Addr` is the data constructor.

Types introduced by a `data` declaration are unique and so are the data constructors introduced with them: the labels used to identify the record type fields are also unique, and may not be reused in other (record) types.

Type aliases also exist in Haskell, to create synonyms for existing types names or expressions: they are introduced by the keyword `type`, as shown below:

```
type Radius  = Double
type Side    = Double
```

Note, however, that unlike `data` declarations, `type` declarations do not introduce any data constructor, hence no new data domain, only a new type name for an existing data domain. In that respect, Haskell `type` declarations are much like `typedef` declarations in C.

Variant Types Haskell variant types are defined by a `data` declaration which introduces the new type constructor and one or more domain constructors, each of which may have zero or more arguments.

```
data Shape   = Circle Radius | Rectangle Side Side
data IntList = Empty | Cons Int IntList
```

As a result of the declaration, the type `Shape` is univocally identified by its name, and the two data constructors `Circle` and `Rectangle` may only be associated with type `Shape`. As discussed in our last lecture, this also solves the typing problem associated with variants.

`IntList` is a further datatype, which illustrates the use of data constructors with no argument, like `Empty` in a recursive definition of a type `IntList` which represents lists of integers: as anticipated, the use of nominal types makes the definition of recursive types particularly simple and natural. Given the declarations above, shapes and integer lists may be expressed simply as follows:

```
let a_circle : Shape = Circle 3.0
let a_list : IntList = Cons 5 (Cons 3 (Cons 4 Empty))
```

ADTs in Scala

Records as (case) classes In Scala, like in other object oriented languages (most notable Java), records and record types are somehow subsumed by objects and classes.

We should note that there is a fundamental difference between objects and records in that methods of objects have access to the objects' structure (other methods and fields) whereas records simply collect fields without providing any cross-referencing mechanism among them. The key to this dichotomy is the implicit parameter `this` that object methods can count on as a primitive mechanism to access the hosting object as a self-reference.

Having said that, recent versions of Java have introduced special constructs to provide for a representation of data structures fully coherent with the typing of records found in other functional

programming languages. Scala also has a construct that severs the purpose: *case classes*,

Case classes are like regular classes with a few special features that make them effective for modelling algebraic datatypes. We illustrate such features with the Scala implementation of the datatypes discussed above.

```
case class Pair(fst: Int, snd : Double)
case class Address(city : String, street :String, number : Int)
```

The two declarations introduce two cases classes whose name serves both as type and as domain constructor. For each case class, Scala automatically generates a primary data constructor so that case class instances may be constructed using the class name as a constructor (without `new`). As a result, the following two declarations introduce, respectively a `Pair` and an `Address`

```
val p = Pair(5,3)
val a = Address("Boston", "Worcester", 15)
```

Case classes have a variety of useful features that make them different from standard classes and construe them as a faithful proxy of records in functional languages: they are immutable, they come with a neat implementation for equality (instances of case classes are always compared structurally), and with a number of additional useful operators and methods.

Most notably, however, as Odersky, Spoon, and Venners note in their Scala book, “the biggest advantage of case classes is that they support pattern matching.”

Variant Types with Enumerations In Scala, variant datatypes may be defined either with `enum` declarations

```
enum Shape:
   case Circle(radius : Double)
   case Rectangle(width : Double, height: Double)

enum IntList:
   case Empty
   case Cons(hd : Int, tl: IntList)
```

As we see, Scala enumerations are very much like datatypes definition in Haskell: the `enum` clause introduces a type name (the constructor of the new type) and the subsequent `case` clauses define the data constructors for the type domain.

As we will show below, `enum` declarations turn out to be just (rather convenient) syntactic sugar for systems of case classes in which the type introduced in the `enum` clause is the common super-type of the types associated with the cases classes that implement the `case` clauses of the enumeration. As a result, a variant type in Scala results in the creations of *many* types -- precisely, one type associated with each clause of the variant plus the variant type itself, with all the former subtypes of the latter. This is conceptually different from what happens in Haskell, in which a variant definition introduces just one type, and the different clauses of the definition introduce the constructors of the sub-domains (as opposed to the subtypes) of the variant type being defined.

The implementation of enumerations with cases classes is almost transparent, except for one detail: to access the data constructors associated with the `enum` declaration, one has to explicitly get them in scope with an explicit *import* declaration.

Given the two enumerations, we may thus use them to build shapes and integer lists as follows:

```
import Shape.{Circle, Rectangle}
import IntList.{Empty, Cons}

val a_circle : Shape = Circle 3.0
let a_list : IntList = Cons(5,Cons(3,Cons(4, Empty)))
```

Pattern matching

We illustrate pattern matching in Haskell as a representative of a mechanism found in many programming languages, pointing to alternatives offered by Scala as appropriate / significant.

Pattern matching in Haskell is an extension of the foundational mechanism illustrated in our last lecture, which accommodates a variety of pattern forms.

A `case` expression

```
case exp of
  p1 -> exp1
  ...
  pn -> expn
```

evaluates the expression `exp` to be matched and then tries each of the patterns `p1` to `pn` in the order they are listed. The first pattern that matches is selected, and the corresponding expression `expi` following the arrow is evaluated: no other match is attempted. If none of the patterns match, a

runtime error is raised. If the cases of a match do not cover all constructors of a datatype, GHC will issue a warning (when compiled with `-Wall` or `-Wincomplete-patterns`).

There exist many kinds of patterns in Haskell other than the variable and constructor patterns we have discussed.

Matching on constant patterns. Constant patterns are simply a special case of constructor patterns. The following code snippet shows some examples:

```
describe :: (Show a) => a -> String  -- not quite: see below
describe x = case x of
  5    -> "five"
  True -> "true"
```

In practice, matching on values of different types within a single function requires a sum type or overloading; Haskell's type system will not allow a single function to match against both `Int` and `Bool` literals directly. A more idiomatic example on a single type:

```
describeBool :: Bool -> String
describeBool True  = "true"
describeBool False = "false"
```

Scala provides more flexible mechanisms, via subtyping:

```
def describe(x: Any): String = x match
  case 5    => "five"
  case true => "true"
  case "hi" => "hello"
  case Nil  => "empty list"
  case _    => "something else"
```

The closest to the above in Haskell would be to introduce a variant type as shown below:

```haskell
data Describable = DInt Int | DBool Bool | DString String | DList [Int]

describe :: Describable -> String
describe (DInt 5)      = "five"
describe (DBool True)  = "true"
describe (DString "hi") = "hello"
describe (DList [])    = "empty list"
describe _             = "something else"
```

Matching on data constructors. This is the primary use of pattern matching in Haskell, and it coincides directly with matching on algebraic data types:

```haskell
data Shape = Circle Double | Rectangle Double Double
area :: Shape -> Double
area (Circle r)      = pi * r ^ 2
area (Rectangle b h) = b * h
```

When all constructors of a datatype are handled, the match is exhaustive; GHC can verify this and warn about missing cases.

Matching on list patterns. Lists support constructor patterns via `[]` and `(:)`:

```haskell
describeList :: [Int] -> String
describeList [0, _, _] = "three elements, starts with 0"
describeList _         = "something else"
```

Unlike Scala's `_*`, Haskell does not have a built-in "rest of any length" pattern within list literal syntax. Instead, the cons pattern naturally handles arbitrary-length lists:

```haskell
startsWithZero :: [Int] -> Bool
startsWithZero (0 : _) = True
startsWithZero _       = False
```

Pattern guards. Patterns in Haskell are *linear*, which means that each variable may occur in a pattern at most once. Pattern guards provide a mechanism to get around this constraint:

```haskell
isSquare :: Shape -> Bool
isSquare (Rectangle b h) | b == h = True
isSquare _                        = False
```

As-patterns. Variables may be bound to the whole value being matched using the `@` syntax. In the pattern `x@p`, the variable `x` is bound to the entire value, while the sub-pattern `p` destructs it:

```
firstAndAll :: [a] -> (a, [a])
firstAndAll xs@(x : _) = (x, xs)
firstAndAll []         = error "empty list"
```

Patterns in bindings. Patterns can be used in `let` and `where` bindings. The following destructs a tuple:

```
let (number, str) = (42, "hello")
```

Function definitions by cases. The most common and idiomatic use of pattern matching in Haskell is in function definitions, where each equation corresponds to a case. The `area` function above is already written in this style. The general form

```
f pattern_11 ... pattern_1k = expr_1
...
f pattern_n1 ... pattern_nk = expr_n
```

is semantically equivalent to a definition using a `case` expression:

```
f x1 x2 ... xk = case (x1, ..., xk) of
  (p_11, ..., p_1k) -> e_1
  ...
  (p_n1, ..., p_nk) -> e_n
```